⚡ FULL-STACK AI DEVELOPMENT

Build Your First AI Chatbot

A Complete Beginner's Guide — React · TypeScript · Node.js · Express · Azure OpenAI

React | TypeScript | Node.js | Express | Azure OpenAI

Ruthran Raghavan · Chief AI Scientist · ruthranraghavan.com

What We Are Building

A real-time AI chatbot where users type messages and receive intelligent AI responses

The Problem

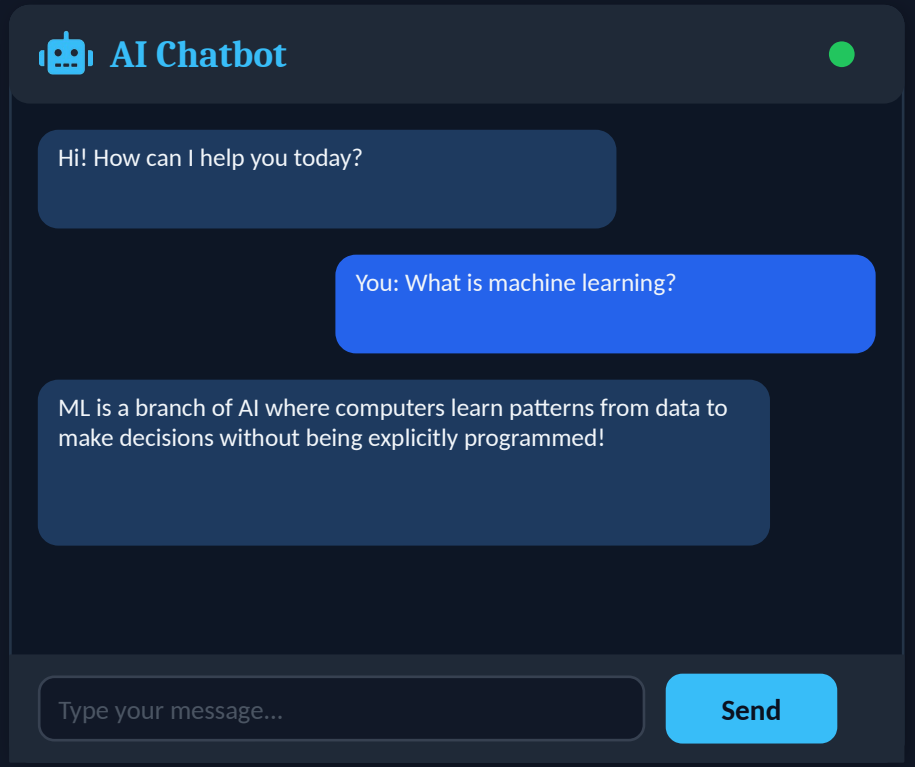
Users need answers instantly, 24/7, without waiting for a human support agent.

The Solution

An AI chatbot powered by Azure OpenAI that responds in natural language — instantly.

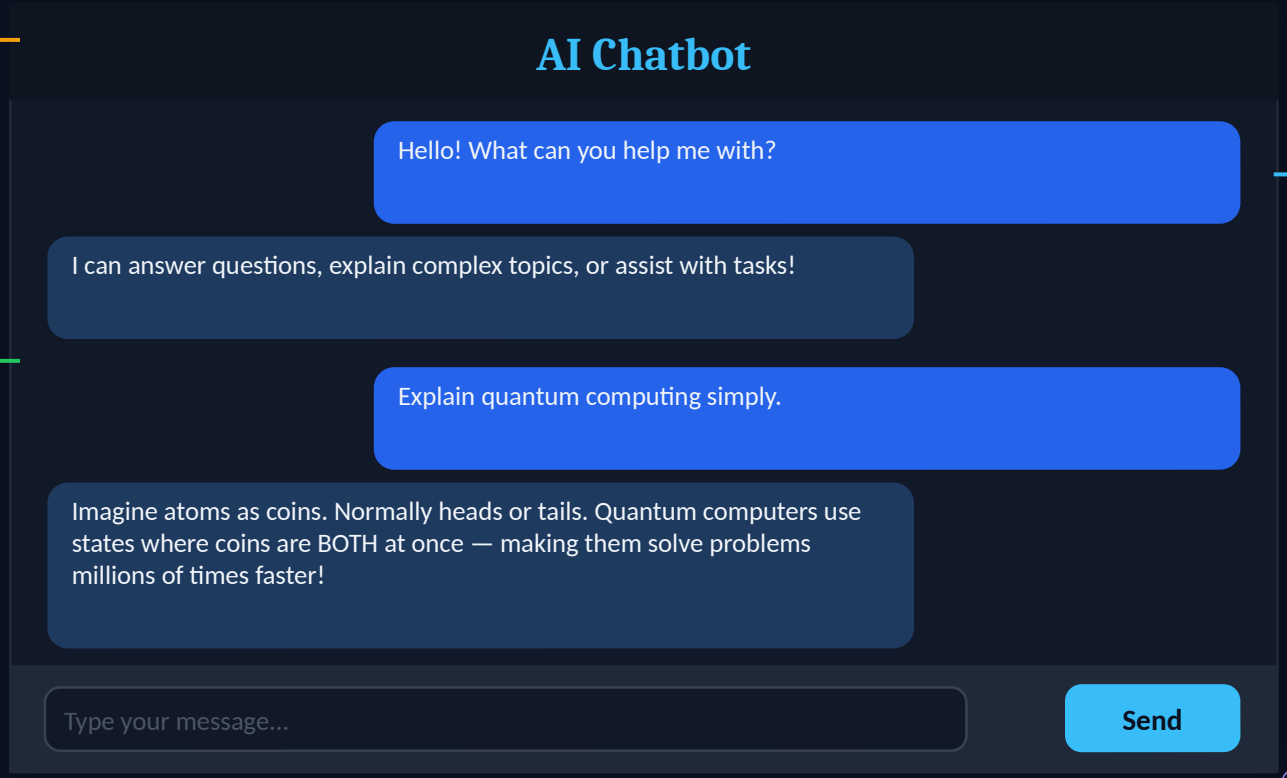
What Our Chatbot Can Do

- → Accept real-time user text messages
- → Send messages to Azure OpenAI for processing
- → Display AI responses in a beautiful chat UI
- → Handle errors gracefully without crashing



Final Application — A Look Inside

Exactly what you will build — a fully-functional dark-themed AI chat interface



Technology Stack

Five technologies — each with one specific job in our AI chatbot

React

v18

Builds the chat UI — the window, message bubbles, and input box

TypeScript

v5

Adds type safety — catches coding mistakes before the app runs

Node.js

v20

Runs JavaScript on the server — powers our Express backend

Express

v4

Creates API routes — the /chat endpoint React calls

Azure OpenAI

GPT-4

The AI brain — generates intelligent replies to user messages

Frontend vs Backend — Why Do We Need Both?

Every web application has two halves — understanding this is fundamental

FRONTEND

React + TypeScript

👁️ Visible	Everything the user sees
🖱️ Interactive	Handles clicks, typing, scrolling
📡 Communicates	Sends data to the backend via HTTP
🌐 Runs In	The user's web browser

POST

JSON

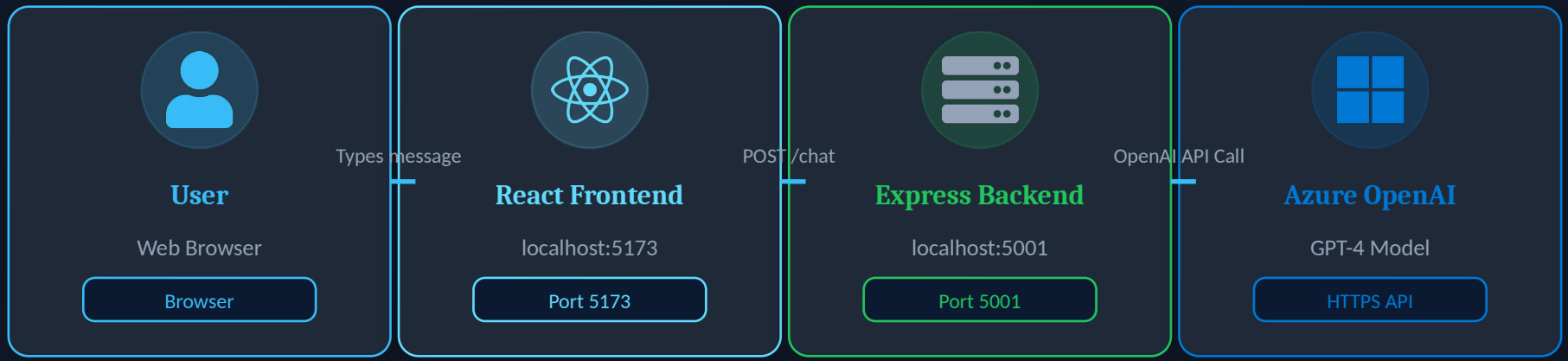
BACKEND

Node.js + Express

🙈 Hidden	Server code users never see
🔐 Secure	Stores API keys safely in .env
🤖 AI Bridge	Calls Azure OpenAI on your behalf
🖥️ Runs On	Node.js server (localhost:5001)

Project Architecture — The Big Picture

Four layers — each one with a specific role in the AI chatbot chain



← AI response flows back: Express → React → User sees the reply

Key Insight:

React NEVER talks to Azure OpenAI directly. All AI calls go through Express. This keeps the API key hidden from users.

Request Flow — Step by Step

The complete lifecycle of one chat message — from typing to seeing the AI reply

1 Browser

User types message & clicks Send

2 React

React captures input → calls sendMessage()

3 React

User message added, loading=true

4 HTTP

POST /chat sent to Express server

5 Express

Express calls Azure OpenAI API

6 Azure

AI generates and returns reply

7 Express

Express sends JSON reply to React

8 React

React renders reply, loading=false

Project Folder Structure

How the code is organized — two separate apps, each with one responsibility

📁 Project Layout

```
📁 day-3/
  📁 backend/
    📄 .env
    📄 .env.example
    📄 package.json
    📄 server.js
  📁 frontend/
    📄 package.json
    📁 src/
      📄 App.tsx
      📄 App.css
      📄 main.tsx
```

⚡ server.js

Express server. Handles /chat endpoint, calls Azure OpenAI, returns replies to React.

🔒 .env

Stores secret credentials. NEVER commit to GitHub! Contains API keys.

⚛ App.tsx

Main React component. All chat UI, state, and event handling lives here.

🎨 App.css

All visual styles — dark theme, message bubbles, chat container, input area.

React Fundamentals

Four core concepts you need before writing React code — explained simply

Components

Reusable UI building blocks. Our entire chatbot is one component (App.tsx). Like custom HTML elements you invent.

🏗️ LEGO bricks — snap together to build bigger things

JSX

HTML + JavaScript mixed together. Write HTML-like syntax inside JS functions. React converts it to real browser HTML.

📝 A smart template that understands logic

State

Data that changes over time. When state changes, React automatically updates the UI. We use useState() to create state.

📊 A scoreboard — when score changes, display updates

Props

Properties passed from parent to child components. Like function parameters for components — data flows downward.

📬 Mail — sent from parent to child with information

Understanding Components

A React component is a JavaScript function that returns JSX (HTML-like code)

Think of it like this:

A component is a custom HTML tag you invent. Instead of , you create <App />. React calls the function, gets the JSX, and renders it in the browser. One function = one piece of UI.

Our App Component Has:

📦 **State**	messages, input, loading
🎯 **sendMessage()**	The AI chat logic function
🖼️ **JSX Return**	Chat box, messages list, input area

```
// App.tsx — Our React Component

export default function App() {
  // 1. STATE: data that can change
  const [messages, setMessages] = useState([])
  const [input, setInput]       = useState("")
  const [loading, setLoading]   = useState(false)

  // 2. FUNCTION: handles the chat logic
  const sendMessage = async () => { ... }

  // 3. JSX RETURN: what the user sees
  return (
    <div className="chat-container">
      <h2>AI Chatbot</h2>
      <div className="chat-box">{ messages }</div>
      <div className="input-area">{ input }</div>
    </div>
  )
}
```

Understanding State (useState)

State is React's memory — when it changes, the UI updates automatically

messages

Type: Message[]

Default: []

Array of all chat messages. Grows each time user or AI sends a message. Each item has { text, sender }.

```
const [messages, setMessages]
  = useState([])
```

input

Type: string

Default: ""

Current text in the input box. Updates on every keystroke. Reset to '' after sending a message.

```
const [input, setInput]
  = useState("")
```

loading

Type: boolean

Default: false

Is the AI thinking? True while waiting. Disables the Send button to prevent duplicate messages.

```
const [loading, setLoading]
  = useState(false)
```

Understanding Event Handling

React listens to user actions and triggers functions in response — in real-time

onChange `<input>` → User is typing

Calls: `setInput(e.target.value)`

When this fires, React runs the handler function immediately

onKeyDown `<input>` → Key pressed → "Enter"

Calls: `sendMessage()`

When this fires, React runs the handler function immediately

onClick `<button>` → Send button clicked

Calls: `sendMessage()`

When this fires, React runs the handler function immediately

```jsx
// Event handlers in our JSX

<input
  value={input}
  onChange={
    (e) => setInput(e.target.value)
  }
  onKeyDown={
    (e) => e.key === "Enter" && sendMessage()
  }
/>

<button
  onClick={sendMessage}
  disabled={loading}
>
  Send
</button>
```

Understanding the Fetch API

How React sends a message to Express and waits for the AI reply — asynchronously

What is fetch()?

fetch() is a built-in browser function that sends HTTP requests. Like a waiter: you place an order (request), it goes to the kitchen (Express), and brings back the food (response). It is async — the page stays interactive while waiting.

1	Call fetch()	Send a POST request to Express with the user message as JSON body
2	await response	Wait for Express. Non-blocking — the browser stays responsive during the wait
3	Parse JSON	response.json() converts the server's text response into a JS object
4	Update State	Add the AI reply to the messages array — UI updates automatically

```
// sendMessage() in App.tsx

const sendMessage = async () => {
  if (!input.trim() || loading) return

  setLoading(true)
  setInput("")

  // Step 1: Call Express backend
  const response = await fetch(
    "http://localhost:5001/chat",
    { method: "POST",
      headers: { "Content-Type":
"application/json" },
      body: JSON.stringify({ message: input })
    }
  )

  // Steps 2+3: Await and parse JSON
  const data = await response.json()

  // Step 4: Update state with AI reply
  setMessages(prev => [
```

TypeScript Basics

JavaScript with type safety — catches bugs before your code runs, not after

Type Annotations

Explicitly tell TS what type a variable holds.

```
let name: string = "Alice"
let age: number = 25
let active: boolean = true
```

Type Inference

TS guesses the type automatically from the value.

```
let name = "Alice"
// TS knows it's a string
name = 42  // ❌ Error: not a number
```

Union Types

A value can be one of several specific types.

```
let sender: "user" | "bot"
sender = "user"   // ✅ OK
sender = "admin"  // ❌ Error!
```

Interfaces

Define the exact shape/structure of an object.

```
interface Message {
  text:   string
  sender: "user" | "bot"
}
```

TS TypeScript catches bugs at WRITE TIME (in your editor with red underlines) — not at RUNTIME when real users see the crash!

Ruthran Raghavan · Chief AI Scientist · ruthranraghavan.com

The Message Interface — TypeScript in Action

Defining the exact shape of every chat message — no guessing, no mistakes

```
// src/App.tsx

// 1. Define the shape of a Message
interface Message {
  text:   string          // The message content
  sender: "user" | "bot" // Only these two values
allowed!
}

// 2. State uses this type
const [messages, setMessages] =
useState<Message[]>([])

// 3. Creating a user message
const userMessage: Message = {
  text:   input,
  sender: "user"  // ✅ TypeScript checks this is
valid
}

// 4. Creating a bot message
const botMessage: Message = {
  text:   data.reply,
```

`text: string`

The actual message content — any string is valid. 'Hello!', 'What is AI?', long responses.

`sender: 'user'|'bot'`

Restricts to ONLY 'user' or 'bot'. Used by CSS to style messages differently: left vs right, different colors.

Used in JSX for styling

className={`message ${msg.sender}`} → applies CSS class "user" or "bot" automatically for correct colors.

Type safety = less bugs

Wrong data type = red underline in editor. Bugs found before users ever see them — 100x better than runtime errors.

Understanding Node.js

JavaScript that runs on the server — not in the browser — enabling our backend

Before Node.js

JavaScript only ran in browsers. Backend required PHP, Python, Java, or Ruby — a completely different language to learn.

What Node.js Does

Runs JavaScript on any computer including servers. Same language, frontend AND backend. Huge productivity boost!

Non-Blocking I/O

Handles thousands of simultaneous requests. While waiting for Azure's AI reply, it serves other users in parallel.

npm Ecosystem

2 million+ free packages. We use express, cors, and dotenv — all installed with one command: npm install.

```
// backend/package.json

{
  "name": "chatbot-backend",
  "type": "module",
  // ^ Enables: import express from "express"
  //   instead of: const express =
require("express")

  "scripts": {
    "start": "node server.js"
    // Run with: npm start
  },

  "dependencies": {
    "express": "^4.19.2",
    // Web framework for our API routes

    "cors": "^2.8.5",
    // Allow React (port 5173) → Express (port
5001)

    "dotenv": "^16.4.5"
```

Understanding Express

Express turns Node.js into a web server — minimal code, maximum power

1 Import & Create

Create a new Express application instance

```
import express from 'express'
const app = express()
```

2 Enable CORS

Allow React (different port) to send requests

```
app.use(cors())
```

3 Parse JSON

Understand JSON in request bodies (req.body)

```
app.use(express.json())
```

4 Define Routes

Create the /chat endpoint React calls

```
app.post('/chat', handler)
```

5 Start Listening

Start server on port 5001

```
app.listen(5001, () => ...)
```

What is a Route?

A route is like a door. When React knocks on the /chat door with a POST request, Express opens it and runs your handler. You can have many routes: /chat, /history, /users — each doing something specific.

Why CORS?

Browsers BLOCK requests between different origins (ports) by default — a security feature. React is on port 5173, Express on 5001. Without app.use(cors()), the browser refuses to let React talk to Express.

Creating the Chat Endpoint

The heart of our backend — POST /chat receives messages and returns AI replies

```
// server.js — The /chat endpoint

app.post('/chat', async (req, res) => {

  // 1. Extract message from React's request
  const { message } = req.body
  if (!message) return res.status(400).json({ error:
'Message required' })

  // 2. Build Azure OpenAI URL using env variables
  const url =
`${process.env.AZURE_FOUNDRY_ENDPOINT}/openai/deployments
/

${process.env.AZURE_FOUNDRY_DEPLOYMENT}/chat/completions
    ?api-version=2024-05-01-preview`

  // 3. Call Azure OpenAI (API key stays on SERVER)
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'api-key': process.env.AZURE_FOUNDRY_KEY,
```

1 `req.body`

Contains the JSON React sent. We extract the 'message' property. Always validate it exists.

2 `Azure URL`

Construct the endpoint URL from environment variables. No hardcoded keys — they're in .env!

3 `'api-key' header`

Azure OpenAI uses 'api-key' in headers (not Authorization). This key is ONLY visible on the server.

4 `choices[0]`

Azure returns multiple completions. We take the first. The reply text is at message.content.

5 `res.json({reply})`

Send the AI reply back to React as JSON. This is what React's await fetch() receives.

Environment Variables — Keeping Secrets Safe

Never hardcode credentials — use .env files to keep API keys out of your code

```
# backend/.env  (NEVER commit this!)

AZURE_FOUNDRY_KEY=sk-abc123...your-actual-key
AZURE_FOUNDRY_ENDPOINT=https://your-
resource.openai.azure.com
AZURE_FOUNDRY_DEPLOYMENT=gpt-4o


# backend/.env.example  (safe to share)

AZURE_FOUNDRY_KEY=your-api-key-here
AZURE_FOUNDRY_ENDPOINT=https://your-
endpoint.openai.azure.com
AZURE_FOUNDRY_DEPLOYMENT=your-deployment-name


# How dotenv loads them into Node.js

import dotenv from 'dotenv'
dotenv.config()  // reads .env file

// Now accessible everywhere in server.js:
const key      = process.env.AZURE_FOUNDRY_KEY
```

NEVER Hardcode Keys

Writing const key = "sk-abc..." in server.js → pushes to GitHub → anyone finds and charges to your account!

.env File Rules

Create .env in backend/. Add .env to .gitignore IMMEDIATELY. Share .env.example with fake placeholder values.

process.env Access

After dotenv.config(), access any variable via process.env.VARIABLE_NAME anywhere in your Node.js code.

Backend Only!

API keys must ONLY exist in the backend. React code is sent to every user's browser — visible in DevTools!

Azure OpenAI Integration

How our Express server communicates with Microsoft's AI model to generate replies

What We Send:

```
// Request body to Azure OpenAI
{
  messages: [
    {
      role: "user",
      content: "What is machine learning?"
    }
  ]
}
```

What Azure Returns:

```
// Azure OpenAI response
{
  choices: [
    {
      message: {
        role: "assistant",
        content: "ML is a branch of AI..."
      }
    }
  ]
}
```

How AI Generates Responses

A beginner-friendly look inside GPT-4 — what actually happens when you ask a question

Tokens

AI doesn't read words — it reads tokens (text chunks, ~4 chars). 'Hello world' = 2 tokens. More tokens = higher cost.

Training Data

GPT-4 was trained on billions of documents. It learned language patterns, facts, reasoning, and how to write code.

Prediction

AI doesn't look up answers. It predicts the most likely next word, building responses token by token, sequentially.

Temperature

Controls creativity. 0 = always same answer. 1 = more varied. We use default (~0.7) for natural conversations.

Key: AI generates text probabilistically. Same question can get slightly different answers each time — that's why it feels natural, not like a database lookup!

Live Chat Walkthrough

Every line of code that executes when you type 'Hello AI' and press Send

1 Browser

User types "Hello AI" → onChange fires

Every keystroke updates input state via setInput(e.target.value)

```
setInput(e.target.value)
```

2 Browser

User presses Enter → onKeyDown triggers

e.key === 'Enter' evaluates to true, sendMessage() is called

```
e.key === 'Enter' && sendMessage()
```

3 React

User message added + input cleared + loading

UI updates immediately — user sees their message appear

```
setMessages([...prev, userMsg])
setInput('')  setLoading(true)
```

4 HTTP

POST /chat request sent to Express

Browser sends HTTP request to localhost:5001

```
fetch("http://localhost:5001/chat",
  { method:"POST", body: JSON })
```

5 Express

Express calls Azure OpenAI with the message

API key sent in header — users CANNOT see this on their end

```
fetch(azureUrl, { headers: {'api-key': key} })
```

6 Express

AI reply received, sent back to React

Express extracts reply from Azure's response and sends JSON

```
res.json({ reply: data.choices[0].message.content })
```

7 React

React renders AI reply + loading = false

Chat updates with bot message. Send button re-enables

```
setMessages([...prev, botMsg])
setLoading(false)
```

Error Handling

What happens when things go wrong — and how we handle it without crashing

```
// App.tsx — try/catch/finally pattern

const sendMessage = async () => {

  setLoading(true)

  try {
    // Code that might fail
    const response = await
fetch('http://localhost:5001/chat', ...)
    const data     = await response.json()

    setMessages(prev => [
      ...prev,
      { text: data.reply, sender: 'bot' }
    ])

  } catch (error) {
    // Runs when ANYTHING in try{} fails:
    // network down, server crash, JSON error
    console.error('Error:', error)
    setMessages(prev => [
      ...prev,
```

Backend Not Running

Forgot to start Express? fetch() fails immediately. catch block shows a friendly error message in the chat.

Wrong API Key

Invalid .env credentials → Azure returns 401. Express catches this and sends status 500 to React's catch block.

Network Issues

If internet drops mid-request, the Promise rejects. The catch block handles this gracefully — no app crash.

The 'finally' Block

ALWAYS runs after try or catch. We set loading=false here — ensuring the button NEVER stays stuck as disabled.

Security Best Practices

Four rules every developer must follow before deploying an AI chatbot

01 Never Call AI APIs from React

✅ Always proxy AI calls through Express

❌ Never call Azure OpenAI from App.tsx directly

ℹ️ React code is sent to every browser. DevTools (F12) can expose any API key you embed in frontend code.

02 Use Environment Variables

✅ Store all secrets in .env files

❌ Never hardcode keys in server.js or any code file

ℹ️ Hardcoded keys end up in Git history — readable by anyone with repo access, even after deleting the file.

03 Validate Input on the Backend

✅ Check: if (!message) return 400 error

❌ Never pass raw user input directly to AI unchecked

ℹ️ Malicious users send huge inputs or try prompt injection to manipulate the AI into doing harmful things.

04 Add Rate Limiting in Production

✅ Use express-rate-limit to cap requests per IP

❌ Never leave /chat open to unlimited requests

ℹ️ Without rate limiting, a script can spam your endpoint 10,000 times/hour — running up your Azure bill!

Common Beginner Mistakes

The 6 most frequent issues in this exact project — and how to fix them fast

1 Calling Azure OpenAI from React

✅ Move ALL AI API calls to Express (backend). The API key must stay server-side only.

2 Only starting ONE server

✅ You need TWO terminals: one for npm run dev (React, port 5173) and one for npm start (Express, port 5001).

3 Mutating state directly

✅ Wrong: messages.push(msg). Right: setMessages(prev => [...prev, msg]). Always use the setter function.

4 Wrong port in fetch() URL

✅ React is on 5173, Express is on 5001. fetch('localhost:5001/chat') — NOT 5173, NOT 3000, NOT 8080.

5 Committing .env to GitHub

✅ Add .env to .gitignore BEFORE your first commit. Check: git status must NOT show .env in the file list.

6 Missing async/await on sendMessage()

✅ fetch() returns a Promise. Without await, you get Promise{} not the data. Always mark functions async.

Debugging Pro Tip:

When something breaks: (1) Open browser DevTools (F12) → Console tab for red errors. (2) Check the Express terminal for server-side error messages. 90% of all bugs in this project are found this way!

How to Extend the Chatbot

Your chatbot is a foundation — these enhancements take it from prototype to product

Chat History

Intermediate

Send all previous messages to Azure OpenAI so the AI remembers earlier context. Real conversations, not just one-shot Q&A.

Authentication

Intermediate

Add user login with Azure AD or Auth0. Each user gets a private session. Only authenticated users can chat.

Database Storage

Intermediate

Save chat history in CosmosDB, MongoDB, or PostgreSQL. Users can return to previous conversations anytime.

System Prompts

Beginner

Give your AI a persona and rules: 'You are a helpful customer service agent for Contoso Corp...' Changes everything!

Streaming Responses

Advanced

Words appear as the AI generates them (like ChatGPT). Uses Server-Sent Events. Dramatically improves perceived speed.

Content Moderation

Advanced

Add Azure Content Safety API to filter inappropriate input/output before processing. Essential for public chatbots.

Enhancement #1: Adding Chat History

Make the AI remember the full conversation — not just the latest message

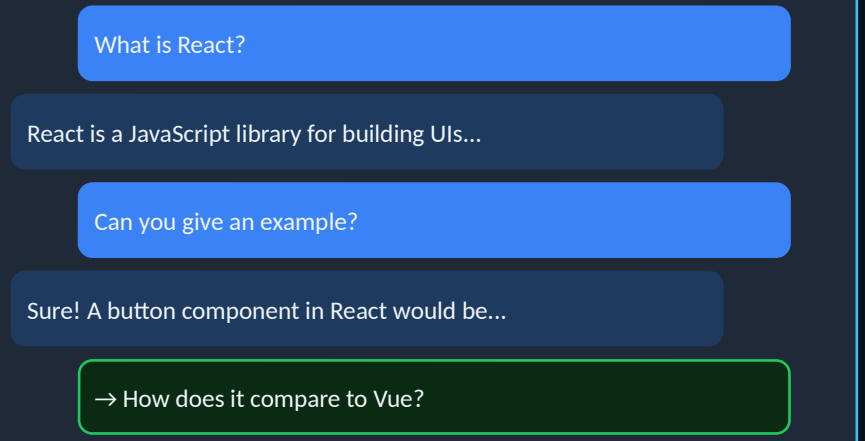
❌ Current — No Memory

```
// Only sends current message
messages: [{ role: 'user', content: message }]
```

✅ Enhanced — With Memory

```
// Maps all messages + adds new one
messages: [
  ...messages.map(m => ({
    role: m.sender === 'user' ? 'user' :
'assistant',
    content: m.text
  })),
  { role: 'user', content: message }
]
```

Why This Matters



ALL previous messages are sent to Azure so the AI understands 'Vue' refers to the conversation about React!

Enhancement #2: Adding Authentication

Know who is chatting — and ensure only authorized users can access your AI

Azure Active Directory (Entra ID)

Enterprise

Enterprise-grade. Users log in with their Microsoft work accounts. Use MSAL.js library with React. Best for corporate applications.

Auth0

Consumer/SaaS

Developer-friendly SaaS authentication. Supports Google, GitHub, Microsoft login. Free tier available. Add to React in minutes with their React SDK.

JWT Tokens

Custom

JSON Web Tokens. After authentication, server issues a signed token. React sends it with every /chat request. Express validates it using jsonwebtoken package.

Typical Auth Flow

1. User clicks 'Login with Microsoft'
2. Azure AD verifies credentials
3. JWT token returned to React app
4. React stores token in memory
5. Every /chat request includes token in header
6. Express validates token before responding
7. Only authenticated users get AI replies

Enhancement #3: Adding Database Storage

Persist chat history so users can return to previous conversations anytime

Azure CosmosDB

NoSQL

Microsoft's globally distributed database. Stores JSON documents. Serverless option available. Perfect native Azure integration. Free tier available.

Best for: Production Azure apps

MongoDB Atlas

NoSQL

Cloud MongoDB with free tier. Stores JSON naturally — our chat messages fit perfectly as document arrays. Great for beginners and rapid prototyping.

Best for: Beginners, prototyping

PostgreSQL (Azure)

SQL

Relational database on Azure. Better when you need complex queries: 'Find all conversations mentioning topic X' or user analytics across sessions.

Best for: Complex queries & analytics

```
// Example MongoDB document — a chat session
{
  _id:       "session_abc123",
  userId:    "user@company.com",
  createdAt: "2025-06-24T10:30:00Z",
```

🎉 Congratulations — You Built an AI Chatbot!

What You Learned Today

Built a React + TypeScript chat UI with useState, event handlers, and Fetch API

Created a Node.js + Express backend with a secure /chat API endpoint

Integrated Azure OpenAI to power real, intelligent AI conversations

Applied security: .env files, backend API proxy, input validation

Understood full-stack architecture: Browser → React → Express → Azure OpenAI

Learned how to extend: chat history, authentication, databases, streaming

🚀 **The best way to learn is to BUILD. Start with this project. Break it. Extend it. Make it yours!**

Ruthran Raghavan · Chief AI Scientist · ruthranraghavan.com